</ Data Mining:
Practical Python
for Beginners

/>

} /> [

Instructor:
Fateme Khodabande

fatemever2001@gmail.com

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Concepts

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Introduction

This chapter introduces the core concepts in Object Orientation. It defines the terminology used and attempts to clarify issues associated with hierarchies. It also discusses some of the perceived strengths and weaknesses of the object-oriented approach. It then offers some guidance on the approach to take in learning about objects.

Classes

- A class is one of the basic building blocks of Python. It is also a core concept in a style of programming known as Object-Oriented Programming (or OOP).
- OOP provides an approach to structuring programs/applications so that the data held, and the operations performed on that data are bundled together into classes and accessed via objects.
- Classes act as templates which are used to construct instances or examples of a class of things. Each example of the class Person has a name, an age, an address, etc., but they have their own values for their name, age and address. For example, to represent the people in a family we might create a class Person with the name Paul, the age 52 and the address set to London. We may also create another Person object (instance) with the name Fiona, the age 48 and the address also of London and so on.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Classes

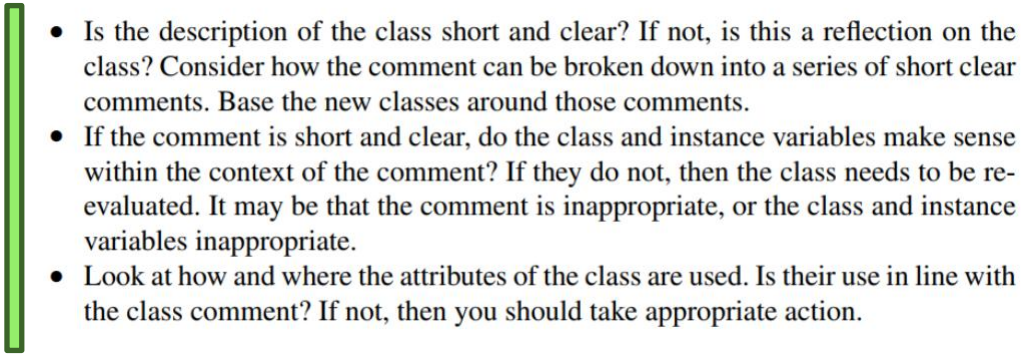
- An instance or object is therefore an example of a class. All instances/objects of a class possess the same data variables but contain their own data. Each instance of a class responds to the same set of requests.
- Classes allow programmers to specify the structure of an object (i.e., its attributes or fields, etc.) and its behavior separately from the objects themselves.
- This is important, as it would be extremely time-consuming (as well as inefficient) for programmers to define each object individually. Instead, they define classes and create instances or objects of those classes.

What Are Classes for?

- We have already seen several types of data in Python such as integer, string, Boolean, etc. Each of these allowed us to hold a single item of data (such as the integer 42 or the string ‘John’ and the value True). However, how might we represent a Person, a Student or an Employee of a firm? One way we can do this is to use a class to represent them.
- As indicated above, we might represent any type of (more complex) data item use a combination of attributes (or fields) and behaviours. These attributes will use existing data types, these might be integers, strings, Booleans, floating-point numbers or other classes.

What Should a Class Do?

- A class should accomplish one specific purpose; it should capture only one idea.
- If more than one idea is encapsulated in a class, you may reduce the chances for reuse, as well as contravene the laws of encapsulation in object-oriented systems.



- Is the description of the class short and clear? If not, is this a reflection on the class? Consider how the comment can be broken down into a series of short clear comments. Base the new classes around those comments.
- If the comment is short and clear, do the class and instance variables make sense within the context of the comment? If they do not, then the class needs to be re-evaluated. It may be that the comment is inappropriate, or the class and instance variables inappropriate.
- Look at how and where the attributes of the class are used. Is their use in line with the class comment? If not, then you should take appropriate action.

Class Terminology

- Class: A class defines a combination of data and behaviour that operates on that data. A class acts as a template when creating new instances.
- Instance or object: An instance also known as an object is an example of a class. All instances of a class possess the same data fields/attributes but contain their own data values. Each instance of a class responds to the same set of requests.
- Attribute/field/instance: variable The data held by an object is represented by its attributes (also sometimes known as a field or an instance variable). The “state” of an object at any particular moment relates to the current values held by its attributes.

Class Terminology

- Method: A method is a procedure defined within an object.
- Message: A message is sent to an object requesting some operation to be performed or some attribute to be accessed. It is a request to the object to do something or return something. However, it is up to the object to determine how to execute that request.

02

Class in Python

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Class Definitions

In Python, a class definition has the following format:

- class nameOfClass(SuperClass):
- __init__
- Attributes
- Methods

Example

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Class Definitions

- The Person class possesses two attributes (or instance variables) called *name* and *age*
- There is also a special method defined called __init__. This is an initializer (aka constructor) for the class. It indicates what data must be supplied when an instance of the Person class is created and how that data is stored internally.
- Let us look for a moment at the special variable self . This is the first parameter passed into any method. However, when a method is called, we do not pass a value for this parameter ourselves; Python does. It is used to represent the object within which the method is executing. This provides the context within which the method runs and allows the method to access the data held by the object. Thus *self* is the object itself)

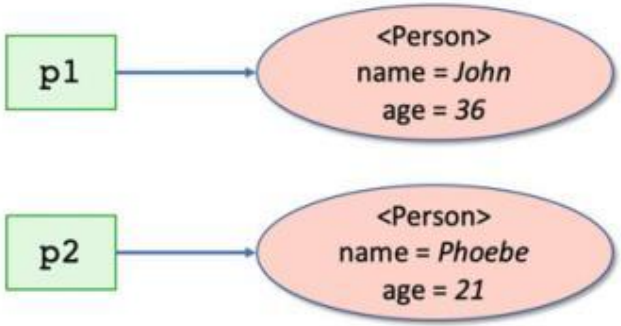
1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Class Definitions

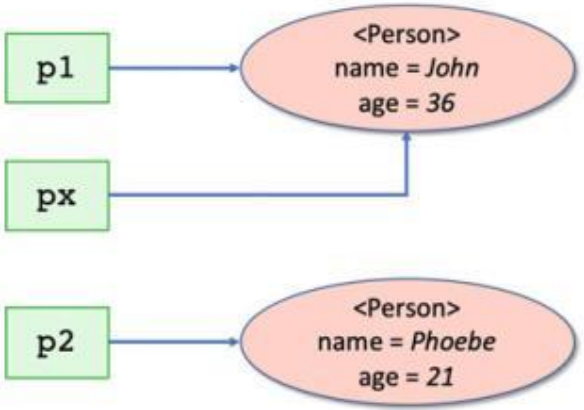
- You may also be wondering about that term *method*. A method is the name given to behavior that is linked directly to the Person class; it is not a free-standing function rather it is part of the definition of the class Person.

Person Class

- Creating Examples of the Class Person



- Be Careful with Assignment



1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 1 0 1

Person Class

Accessing Object Attributes / Defining a Default String Representation

- `__str__()` is intended dot be used to create a string version of an object that can be used for printing and logging purposes.
- `__repr__()` is intended to generate a string representation of an object that can be used to recreate the object - hence the format used above that would allow you to use `Person(name = John, age = 36)` to recreate an instance of the person John.

Which is used by Python depends on what you have define. If you print an object out then Python first tries to use the `__str__()` method, if that does not exist it uses the `__repr__()` method. You can therefore choose to just use the `__repr__()` method. However, if instances of a class are contained within for example a list, then when the list is printed out it will always use the `__repr__()` method. For this reason it is quite common to find that a class has both a `__str__()` and a `__repr__()` method using slightly different formats. Alternative, one of the two methods may call the other one.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Person Class

- Adding a Birthday Method
- The Del Keyword
- Automatic Memory Management
- Intrinsic Attributes

- `__name__` the name of the class
- `__module__` the module (or library) from which it was loaded
- `__bases__` a collection of its base classes (see inheritance later in this book)
- `__dict__` a dictionary (a set of key-value pairs) containing all the attributes (including methods)
- `__doc__` the documentation string

For objects:

- `__class__` the name of the class of the object
- `__dict__` a dictionary containing all the object's attributes

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Operator Overloading

Operator	Expression	Method
Addition	q1 + q2	__add__(self, q2)
Subtraction	q1 - q2	__sub__(self, q2)
Multiplication	q1 * q2	__mul__(self, q2)
Power	q1 ** q2	__pow__(self, q2)
Division	q1 / q2	__truediv__(self, q2)
Floor Division	q1 // q2	__floordiv__(self, q2)
Modulo (Remainder)	q1% q2	__mod__(self, q2)
Bitwise Left Shift	q1 << q2	__lshift__(self, q2)
Bitwise Right Shift	q1 >> q2	__rshift__(self, q2)

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

Operator Overloading

Operator	Expression	Method
Less than	`q1 < q2`	`__lt__(q1, q2)`
Less than or equal to	`q1 <= q2`	`__le__(q1, q2)`
Equal to	`q1 == q2`	`__eq__(q1, q2)`
Not Equal to	`q1 != q2`	`__ne__(q1, q2)`
Greater than	`q1 > q2`	`__gt__(q1, q2)`
Greater than or equal to	`q1 >= q2`	`__ge__(q1, q2)`

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 1 1 1 0 1

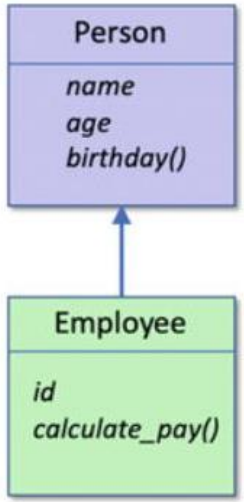
Operator Overloading

```
class Quantity:
        def __init__(self, value=0):
            self.value = value
        def __add__(self, other):
            new_value = self.value + other.value
            return Quantity(new_value)
        def __sub__(self, other):
            new_value = self.value - other.value
            return Quantity(new_value)
        def __str__(self):
             return 'Quantity[' + str(self.value) + ']'
```

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

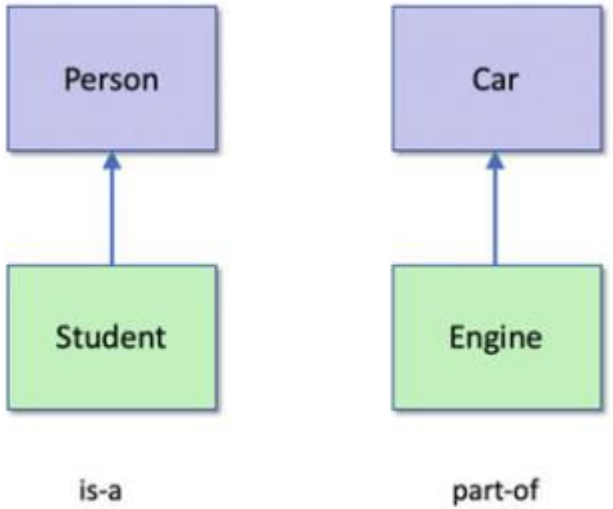
Inheritance

- Inheritance allows features defined in one class to be inherited and reused in the definition of another class.
- We might then decide that we want to have another class Employee and that employees also have a name and an age and will have birthdays. However, in addition an Employee may have an employee Id attribute and a calculate_pay() behavior



Inheritance

- `class SubClassName(BaseClassName):`
  `class-body`
- `super().__init__()` Keyword
- is-a vs part-of



Exercise:

Write a class **DataSet** that takes a list of mixed values and keeps only numbers.

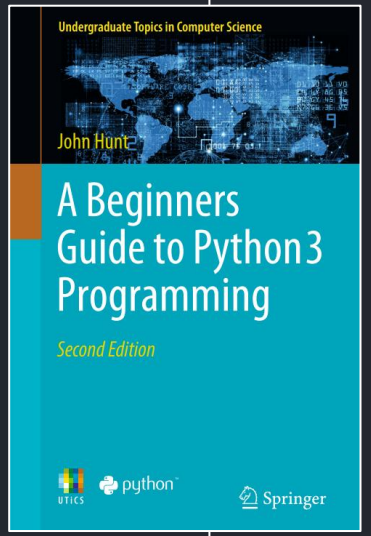
```
raw_data = [10, 45, 90, -3, None, 120, 33, 45, "x", 50]
```

Your class must:

- **__init__** : Store only integers and floats from the input list.
- **add(value)** : Add the value if it's numeric; otherwise print "Invalid value!".
- **clean()** : Remove all numbers outside the range **0–100**.
- **summary()** :
  If no data exists, return "No valid data".
  Otherwise return a dictionary with: count, mean, min, max
- **__str__** : Return the dataset as a readable string.

1 0 1 1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 0 1 1 1 1 1 0 1

</ Reference: A Beginners Guide to Python 3 Programming />



https://doi.org/10.1007/978-3-031-35123-8

1 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 1 0 1 1 0 1 1 0 1 1 1 0 1 1 0 1 1 1 0 1